

Student Record Management System

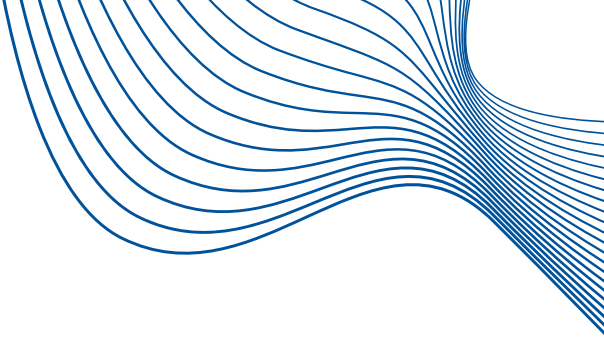
Group Coursework Project

Prepared by:

- Layan Saleh Al-harbi
- Ghand Al-habib
- Danah Alolayan
- joud Almatrafi



System Design & Architecture



Project Overview

The Student Record Management System (SRMS) is a Java-based application designed to manage student records using Object-Oriented Programming principles. The system organizes data in a structured way to improve readability and maintainability.

System Architecture

The system is built using multiple classes, where each class has a specific responsibility. This modular design makes the system easier to develop, maintain, and extend.

System Functionalities

The system supports key operations such as adding students, displaying records, validating GPA, saving and loading data from files, and generating reports based on GPA.

It also uses multithreading to perform automatic background saving, which helps protect data without interrupting the user.

Main Components

The system consists of the following main classes:

- Person, Student, Address → represent student data
 - StudentManager → manages student operations
 - FileHandler → handles file saving and loading
 - AutoSaveThread → performs background saving
 - ReportGenerator → generates GPA reports
 - Main → controls program flow
- 

Classes & Methods

Classes & Methods

The system is built using several classes, each with specific methods:

Person Class.

- Abstract class containing common attributes (name, age)
- Defines abstract method `getDetails()`

Student Class

- Extends Person
- Stores ID, GPA, department, and address

Methods:

- `getId()`
- `getGpa()`
- `getDepartment()`
- `getDetails()`
- `toString()`

StudentManager Class

- `addStudent(Student s)`
- `removeStudent(String id)`
- `getStudents()`

FileHandler Class

- `saveToFile(List<Student>)`
- `loadFromFile()`

AutoSaveThread Class

- `run()` method for periodic saving

ReportGenerator Class

- `run()` method for generating GPA-based reports

OOP Principles

The system applies key Object-Oriented Programming principles:

Inheritance

Student class inherits from Person to reuse common attributes.

Encapsulation

All attributes are private or protected and accessed through methods.

Polymorphism

getDetails() is overridden in Student to provide specific implementation.

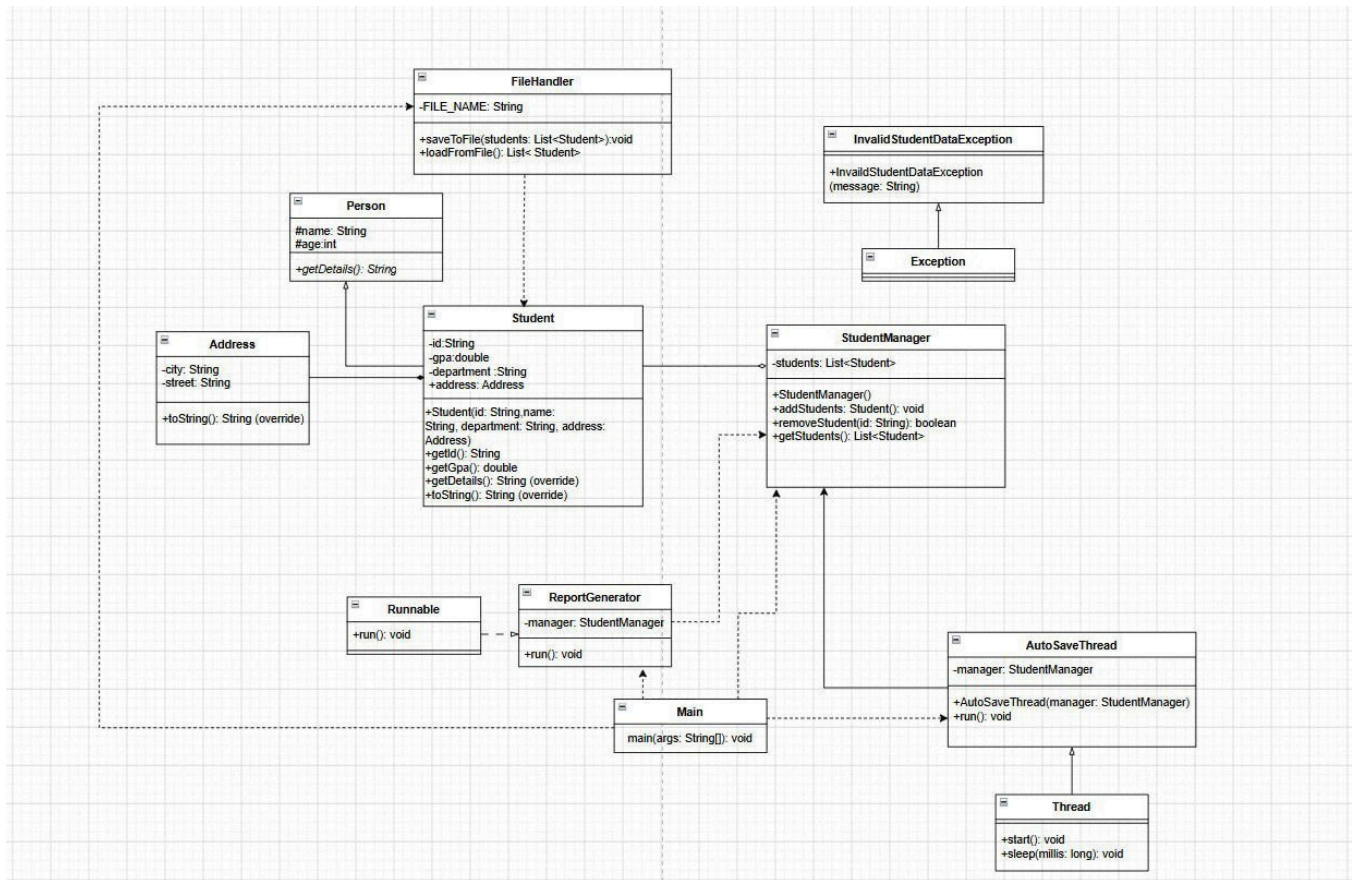
Abstraction

Person class contains abstract method getDetails() to enforce implementation in subclasses.

Composition

Student class contains Address object, representing a "has-a" relationship

UML Class Diagram



File Handling & Multithreading

The system uses file handling to store and retrieve student data.

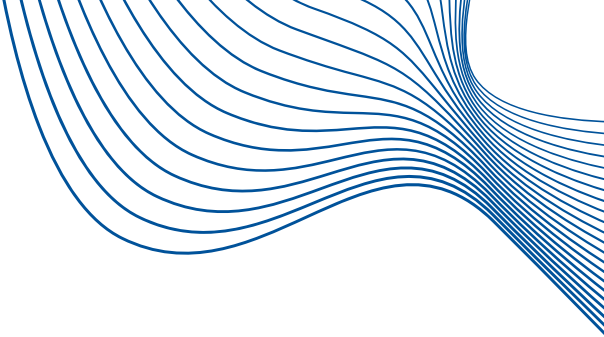
File Handling (FileHandler)

- Saves student data into "students.txt"
- Loads data when the program starts
- Uses `BufferedReader` and `FileWriter`

Multithreading

- `AutoSaveThread` automatically saves data every 30 seconds
- `ReportGenerator` runs separately to generate GPA reports
- Improves performance and prevents blocking of main program

Challenges & GitHub



Challenges Encountered

- Handling file reading and writing safely
- Managing invalid GPA input values
- Ensuring smooth multithreading without conflicts

These were solved using exception handling, synchronization, and structured class design.

GitHub Collaboration

The project was developed collaboratively using GitHub for version control and teamwork.

We organized the project by placing each class in a separate File for better structure and clarity. You can access the project files

and README using the Following link:

<https://github.com/joudalmatrafi/OOP-project>

Ethics & Future Improvements

Ethical Considerations

The system follows ethical principles to ensure proper handling of student data:

- Encapsulation is used to protect sensitive information and restrict direct access to data.
- Unauthorized access to student records is prevented through controlled methods.
- Student data is stored and managed securely to maintain privacy and data integrity.

Future Improvements

The system can be improved in the future by enhancing its functionality and performance. Possible upgrades include:

- Replacing file storage with a database system (such as MySQL) for better data management and scalability.
- Adding a graphical user interface (GUI) to improve user experience and usability.
- Improving the search functionality to allow faster and more advanced filtering of student records.
- Enhancing the reporting system with more detailed and visual analytics.

The End

Thank You